



EVENT-DRIVEN ARCHITECTURE (EDA)

EDA

Event Driven

- System components communicate via events
 - Asynchronous messages signaling that something has happened.
 - Instead of direct calls, components react to events, enabling loose coupling, scalability, and real-time processing.
-

KEY CONCEPTS

- Event
 - A notification of a state change (e.g., StudyReceived, ImageReceived).
 - Contains data about what happened but does not dictate what to do next.
 - Event Producer (Publisher)
 - Generates and emits events (e.g., a DICOM File is received - InstanceReceived).
 - Event Consumer (Subscriber)
 - Listens for events and reacts (e.g., Instance Processed and Saved into the DB).
 - Event Channel/Message Broker
 - Transports events between producers and consumers (e.g., Kafka, RabbitMQ, AWS SNS/SQS).
 - Event Bus
 - A middleware that routes events to the right consumers (e.g., Apache Kafka, Azure Event Grid).
-

TYPES OF EVENT-DRIVEN ARCHITECTURES

Type	Description	Use Case
Simple Event Processing	Immediate reaction to events (e.g., sending an email on user signup).	Notifications, logging
Event Stream Processing	Continuous processing of event streams (e.g., fraud detection in transactions).	Real-time analytics, IoT
Complex Event Processing (CEP)	Detects patterns across multiple events (e.g., "3 failed logins in 5 minutes").	Fraud detection, monitoring

ADVANTAGES / DISADVANTAGES OF EDA

ADVANTAGES

✓ **Loose Coupling** –
Producers &
consumers don't need
to know about each
other.

✓ **Scalability** –
Consumers can process
events independently
(horizontal scaling).

✓ **Real-Time
Responsiveness** –
Immediate reaction to
events.

✓ **Fault Tolerance** –
Events can be retried if
a consumer fails.

✓ **Flexibility** – New
consumers can be
added without
modifying producers.

DISADVANTAGES

✗ **Complex
Debugging** – Tracing
event flows can be
challenging.

✗ **Event Ordering
Issues** – Events may
arrive out of order
(requires sequencing
logic).

✗ **Overhead** –
Requires message
brokers and
monitoring.

✗ **Eventual
Consistency** –
Systems may not be
immediately
consistent.

WHEN TO USE EVENT-DRIVEN ARCHITECTURE?

✓ **Real-Time Systems** (e.g., stock trading, IoT sensor data).

✓ **Decoupled Microservices** (services need to communicate without direct API calls).

✓ **Asynchronous Workflows** (e.g., order processing, notifications).

✓ **High-Scalability Needs** (e.g., social media feeds, ride-sharing apps).

COMPARISON WITH LAYERED ARCHITECTURE

Feature	Layered Architecture	Event-Driven Architecture
Coupling	Tight (layers depend on each other)	Loose (services communicate via events)
Scalability	Vertical scaling	Horizontal scaling
Responsiveness	Synchronous (blocking calls)	Asynchronous (non-blocking)
Use Case	Monoliths, CRUD apps	Real-time systems, microservices
